AAVA 2.0
DAST & Penetration Testing Report

Document Control

Customer	Ascendion
Title	AAVA 2.0 (ElderWand) DAST & Penetratiion Testing Report
Document Owner	Ascendion – Security Team
Author	Prasad Maladkar
Reviewer	Navakumari R
Initiation Date	17/10/2025
Document ID	
Version	3.0
Document Status	Release

Proprietary & Legal Notice

This Security Testing Report is the proprietary property of Ascendion and contains confidential information protected under applicable intellectual property and copyright laws. The contents of this report are intended solely for the use of the authorized recipient and Ascendion's internal teams.

Disclosure, distribution, reproduction, or use of this report or any part thereof by any unauthorized party is strictly prohibited. Nothing in this report shall be construed as granting any license or rights, whether expressly or implicitly, to use any of Ascendion's intellectual property.

The Ascendion Terms and Conditions of Supply shall apply to all services referenced herein. In the event of any inconsistency, the terms of this report shall prevail.

Copyright © Ascendion 2025. All Rights Reserved.

Disclaimer

This report is provided "as is" for informational and risk-assessment purposes only. While reasonable effort has been made to ensure the accuracy of the findings, Ascendion expressly disclaims any liability for errors, omissions, or consequences arising from the use or reliance on this report by any party.

This report does not constitute an endorsement or warranty of any product, system, or service described herein. It is the recipient's responsibility to evaluate the risks and take appropriate mitigation actions.

Confidentiality

This report has been prepared exclusively for the use of Ascendion's internal DevSecOps team and designated stakeholders. The content is classified as confidential and must not be disclosed, shared, or reproduced in any form without the prior written approval of Ascendion.

Contents

Disclaimer 3
Confidentiality 3
1 Executive Summary 5
1.1 Summary of vulnerabilities 5
2 Introduction 7
2.1 Background & Scope 7
2.2 Targets 7
2.3 Constraints & limitations 7
2.4 Testing period 7
3 Approach & Methodology 8
4 Risk Ratings 8
4.1 Impact, Likelihood and Overall Risk 8
4.2 Rating Descriptions 9
5 Detailed Findings from Web Application Testing 10
5.1 Risk Findings 10
5.2 Conclusion 52

1 Executive Summary

This Security Testing Report provides a comprehensive assessment of the AAVA 2.0 (ElderWand) application security posture, conducted by the Ascendion AAVASecurity team. The objective of this assessment was to identify security vulnerabilities, evaluate potential risks, and recommend mitigation strategies to strengthen the overall security of the Application.

The assessment was performed using a combination of automated and manual testing tools to detect common and critical vulnerabilities as defined by the OWASP Top 10, and Ascendion's internal security benchmarks.

1.1 Summary of vulnerabilities

Summary of vulnerabilities and table Title:

#	Observation	Risk Rating	Status	Issue ID
1	Exposure of Environment Access Keys in JavaScript Files	Critical	Open	AVA001-20102025-1
2	Application UI Accessible Using Expired Token	Critical	Open	AAVA003-20102025-2
3	Reflected Cross-Site Scripting (XSS) in GET Parameter	Critical	Open	AAVA003-20102025-3
4	Insecure Communication – HTTP Enabled Instead of HTTPS	Critical	Open	AAVA003-20102025-4
5	Privilege Escalation (Unauthorized Admin Portal Access via Response manipulation)	Critical	Open	AAVA003-20102025-5
6	Privilege Escalation — Admin API Accessible Using Normal User Token & Role Update	Critical	Open	AAVA003-20102025-6
7	AI Agent Generates Insecure Code and Secure Code Validator Fails to Detect Vulnerabilities	Critical	Open	AAVA003-20102025-7
8	Account Takeover via Insecure Token Generation and Missing Email Validation	Critical	Open	AAVA003-20102025-8
9	Bypass of Admin-Approval via Response Manipulation (Unauthorized Self-Enrolment)	Critical	Open	AAVA003-20102025-9
10	Application Allows Login Without Authentication (Guest User Access)	High	Open	AAVA003-20102025-10
11	Insecure Storage and Usage of Windows Authorization Credentials in JS File	High	Open	AAVA003-20102025-11
12	Brief Exposure of Authenticated Page Before Login Redirect	High	Open	AAVA003-20102025-12
13	Open Redirect via URL Parameters	High	Open	AAVA003-20102025-13
14	Improper Input Validation	Medium	Open	AAVA003-20102025-14
15	Session Management Vulnerability: (Improper Login Policy Implementation)	Medium	Open	AAVA003-20102025-15
16	Insecure Direct Object Reference (IDOR) via Unrestricted ID Parameter	Medium	Open	AAVA003-20102025-16
17	Concurrent Session Allowed	Medium	Open	AAVA003-20102025-17
18	Clickjacking Vulnerability – Missing Frame Protections	Medium	Open	AAVA003-20102025-18
19	Use of Vulnerable or Outdated Component – DOMPurify v3.1.7	Medium	Open	AAVA003-20102025-19
20	Multiple Identical Tokens Stored in Cookies	Medium	Open	AAVA003-20102025-20
21	Insecure HTTP Methods Allowed (PUT and DELETE)	Low	Open	AAVA003-20102025-21
22	Insecure Cookie Configuration	Low	Open	AAVA003-20102025-22
23	Cross-Origin Resource Sharing (CORS)	Low	Open	AAVA003-20102025-23

Table 1 – Summary of vulnerabilities

#	Observation	Risk Rating	Status	Issue ID
24	Missing preload Directive in HTTP Strict Transport Security (HSTS) Header	Low	Open	AAVA003-20102025-24
25	X-XSS-Protection Header Disabled	Low	Open	AAVA003-20102025-25
26	Sensitive Information Stored in Cookies	Low	Open	AAVA003-20102025-26
27	Missing Content Security Policy (CSP) Header	Low	Open	AAVA003-20102025-27
28	Improper Error Handling with Source Code Disclosure	Low	Open	AAVA003-20102025-28
29	ETag Header Is Not Disabled	Informational	Open	AAVA003-20102025-29

2 Introduction

2.1 Background & Scope

This Web Application Security Testing assessment (DAST) was conducted to identify security flaws in the application's development lifecycle. The scope included Penetration Testing of the application's modules, configuration files, and third-party dependencies.

2.2 Targets

S.No	Project Name	File Name
01	AAVA 2.0 (ElderWand)	https://aava-uat.avateam.io/console/login

Table 2 – Scope

2.3 Constraints & limitations

- Many of the modules are still not configured/working as per expectation.
- Findings are based on the available modules at the time of assessment; any missing or incomplete module was not reviewed.
- Third-party libraries, binaries, and compiled components were excluded unless explicitly included in the scan scope.
- Testing with only one credentials.

2.4 Testing period

- PT activity was conducted from 14th Oct 2025 to 17th Oct 2025.

3 Approach & Methodology

The penetration testing assessment was conducted using a controlled and authorized approach to simulate real-world attack scenarios against the target environment. The methodology followed industry's best practices and OWASP guidelines, focusing on identifying security weaknesses through reconnaissance, vulnerability analysis, and controlled exploitation. The objective was to evaluate the effectiveness of existing security controls without causing disruption to production systems.

Key Steps Involved:

- **Pre-Engagement (Planning & Recon):**
  Define scope, gather target intelligence, and set testing rules.
- **Assessment (Scanning & Manual Verification):**
  Identify vulnerabilities through automated and manual testing.
- **Post-Manual Verification (Reporting & Remediation):**
  Document findings, assess risk, suggest fixes, and support retesting.

4 Risk Ratings

4.1 Impact, Likelihood and Overall Risk

Vulnerabilities are assigned an Impact and Likelihood rating as follows, where Impact considers the vulnerability in isolation and Likelihood considers how likely it is that an attacker would be able to exploit the vulnerability. These ratings are combined to show the overall risk posed by the vulnerability.

Impact	Critical	Medium	High	Critical	Critical
	High	Medium	Medium	High	Critical
	Medium	Low	Medium	Medium	High
	Low	Low	Low	Medium	Medium
		Low	Medium	High	Certain
Likelihood					

Table 3 - Risk Ratings

4.2 Rating Descriptions

Rating	Vulnerability Impact	Likelihood of Exploitation
Critical	• Evidence of an actual security breach was found requiring immediate attention. • A vulnerability exists that provides administrator-level access and for which there is significant recent evidence of exploitation, e.g., by a worm, virus, or hacker/cracker. • An attacker can impersonate another user and perform significant activities as them, e.g., committing fraud. Denial of Service (DoS) attack against a business-critical target that has significant further ramifications, such as endangering human life or National security.	• Evidence of an actual security breach was found requiring immediate attention. • Evidence of a Trojan Horse or worm program that may attack other machines on this or other networks. • Critical DoS attack that is exposed to attack and could reasonably be launched by an anticipated threat.
High	• Large amount of leakage of key information that is likely to lead to interactive access, or DoS, qualifying for High or Critical severity. • An attacker can directly obtain administrator-level access or modify data. An attacker can obtain interactive non-administrator-level access through a commonly used exploit, for which little technical knowledge is required, and which is commonly used as part of a chain of attacks resulting in privilege escalation. • An attacker can impersonate another user and gain significant access to information (e.g., in breach of privacy laws) or perform activities; not qualifying for Critical. Denial of Service (DoS) attack against a business-critical target.	• High severity vulnerability that is easily exploitable by an attacker with little technical knowledge, e.g., the exploit is simple, or an exploit script is freely available from public domain sources. • Critical vulnerability for which there is no particular evidence of exploitation. Denial of Service (DoS) attack against a business-critical target that is known to be achievable with moderate or little technical knowledge.
Medium	• Moderate or high level of information leakage that may be of use to an attacker. • An attacker can gain unauthorized access to the target; not qualifying for High or Critical. • An attacker can impersonate a user; not qualifying for High or Critical. • Vulnerable to a Denial of Service (DoS) attack not qualifying for High or Critical.	• Medium severity vulnerability that is directly exposed to potential attackers. • Higher severity vulnerability that requires a significant degree of technical expertise, e.g., an exploit script is not freely available from public domain sources. Denial of Service (DoS) attack not qualifying for High that is known to be achievable with moderate or little technical knowledge, e.g., is simple to perform or an exploit script is freely available from public domain sources.
Low	• The vulnerability provides some information that may be of use to an attacker. • Vulnerable to a Denial of Service (DoS) attack where the outage or restriction in service is of little or no significance, e.g., only affects an unused service.	• Exploitation is possible but does not qualify for Medium or High.

Table 4 - Rating Descriptions

5 Detailed Findings from Web Application Testing

5.1 Risk Findings

WASA 01	Exposure of Environment Access Keys in JavaScript Files	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The application exposes sensitive environment or API keys in publicly accessible JavaScript files, allowing anyone to view them through the browser.		
Security Impact	Attackers can misuse the exposed keys to access backend services, perform unauthorized actions, or escalate privileges, potentially leading to data compromise or service abuse.		
Recommendation	Move all sensitive keys to server-side environment variables. Never include secrets in client-side code or publicly accessible files. Implement proper access controls and rotate any exposed keys immediately.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html?utm_source=chatgpt.com		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	All env.js files are impacted with sensitive data expose		
Issue Status	**Open**		

Evidence:

Figure 01: accessskey in js file.

WASA 02	Application UI Accessible Using Expired Token	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The application allows users to access the UI even after the authentication token has expired. Session or token validity is not properly verified on the client or server side		
Security Impact	Attackers can reuse expired tokens to maintain unauthorized access, leading to potential session hijacking or exposure of sensitive information.		
Recommendation	Implement strict server-side token validation and enforce automatic logout or re-authentication once a token expires. Invalidate tokens immediately upon expiry or logout.		
References	https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/		
OWASP category	A07:2021 – Identification and Authentication Failures		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

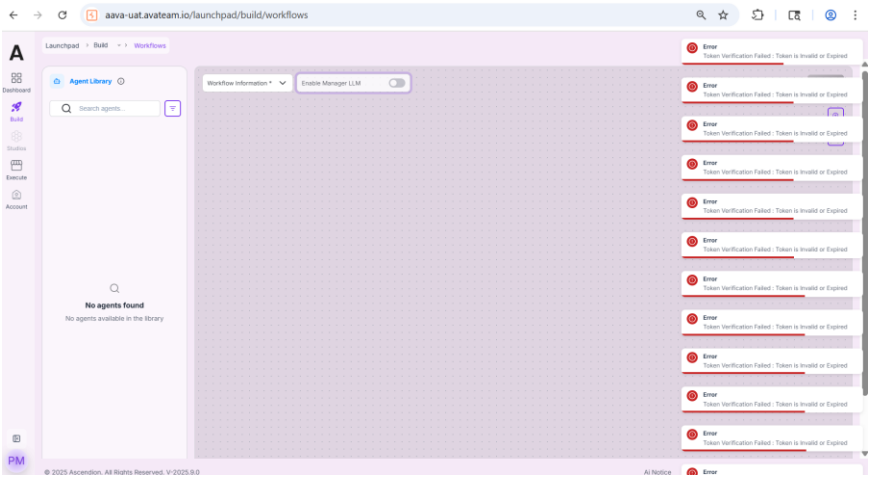


Figure 02: Application UI Accessible Using Expired Token

WASA 03	Reflected Cross-Site Scripting (XSS) in GET Parameter	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The application reflects unsanitized user input from the GET request into the response without proper encoding. For example, requesting: GET /api/auth/basic/loging0wpb%3cscript%3ealert(1)%3c/script%3ewiwsk executes JavaScript in the victim's browser		
Security Impact	An attacker can craft malicious URLs that execute arbitrary JavaScript in the user's browser, leading to session hijacking, credential theft, data manipulation, or redirection to malicious sites.s		
Recommendation	• Sanitize and validate all user input. • Apply output encoding (HTML/JavaScript context-aware escaping) before rendering input in the response. • Implement Content Security Policy (CSP) to mitigate XSS exploitation.		
References	https://owasp.org/www-community/attacks/xss/?utm_source=chatgpt.com		
OWASP category	A3:2021 – Injection		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

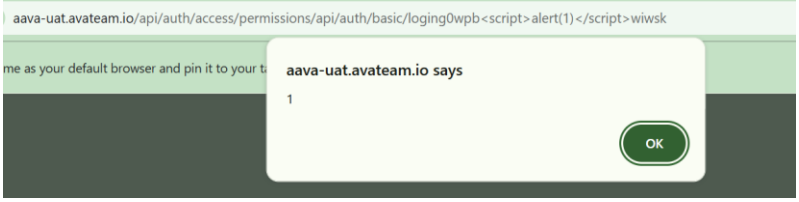


Figure 03: Access https://aava-uat.avateam.io/api/auth/access/permissions/api/auth/basic/loging0wpb%3cscript%3ealert(1)%3c/script%3ewiwsk.

WASA 04	Insecure Communication – HTTP Enabled Instead of HTTPS	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The application is accessible over HTTP, allowing data to be transmitted in plaintext without encryption.		
Security Impact	Attackers on the same network can intercept or modify traffic (MITM attack), potentially leading to credential theft, data leakage, or session hijacking.		
Recommendation	Enforce HTTPS by disabling plain HTTP, enabling HSTS (HTTP Strict Transport Security), and ensuring all sensitive data is transmitted securely.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html		
OWASP category	A02:2021 – Cryptographic Failures		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

```
curl -v http://aava-uat.avateam.io/assets/env.js
* Host aava-uat.avateam.io:80 was resolved.
* IPv6: (none)
* IPv4: 9.223.203.209
*   Trying 9.223.203.209:80...
* Connected to aava-uat.avateam.io (9.223.203.209) port 80
* using HTTP/1.x
> GET /assets/env.js HTTP/1.1
> Host: aava-uat.avateam.io
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 14 Oct 2025 06:50:01 GMT
< Content-Type: application/javascript
< Content-Length: 2564
< Connection: keep-alive
< Last-Modified: Fri, 10 Oct 2025 11:13:57 GMT
< Vary: Accept-Encoding
< ETag: "68e8ea75-a04"
< Expires: Wed, 14 Oct 2026 06:50:01 GMT
< Cache-Control: max-age=31536000
< Cache-Control: public, immutable
< Vary: Accept-Encoding
< Accept-Ranges: bytes
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
< Access-Control-Allow-Methods: GET, HEAD, POST, PUT, DELETE,PATCH, OPTIONS
< Access-Control-Allow-Headers: DNT,X-Ava-Access-Token,X-Ava-Refresh-Token,Keep-Alive,User-Agent,X-Requested-With,If-Modified-Since,C
ache-Control,Content-Type,Authorization,Content-Disposition,Pragma,Accept,Accept-Language
< Access-Control-Expose-Headers: X-Ava-Access-Token,X-Ava-Refresh-Token,Content-Disposition,X-Access-Token
< Access-Control-Max-Age: 43200
<
```

Figure 04: Insecure Communication – HTTP Enabled Instead of HTTPS.

WASA 05	Privilege Escalation (Unauthorized Admin Portal Access via Response manipulation)	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The application relies on client-controllable response values (or responses lacking integrity checks) to determine privilege level; by modifying server responses/parameters (e.g., changing role=default → role=admin or tampering with returned JSON), a normal user can access the admin portal.		
Security Impact	An attacker can escalate privileges from a normal user to admin, allowing access to sensitive admin functionality — modify settings, view/export sensitive data, create/delete accounts, or fully compromise the system.		
Recommendation	• Enforce server-side authorization for every request; never trust client-supplied or client-modified response values for privilege decisions. • Derive roles/privileges from trusted server-side identity claims (session store or signed tokens) and validate them on each protected endpoint. • Implement object-level access control (verify resource ownership) and deny-by-default.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Broken_Access_Control_Cheat_Sheet.html		
OWASP category	A01:2021 — Broken Access Control		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:


```
GET /api/auth/user/details HTTP/2
Host: aava-uat.avateam.io
Cookie: login_type=sso; access_token=
eyJ0eXAi0iJKV1QiLCJhbGci0iJSUzI1NiJ9.eyJzdWIi0iJxcm5ZbmV3RFBp
dkp4ZVhyYzB3emtEQmtCbCNHNmRSR2xLaHRxQXhESzcvIiwiaWF0IjoxNzYwN
DIyMDcyLCJleHAi0jE3NjA0MjYzNzQsImF1ZCI6IjAwMDAwMDAzLTAwMDAtMD
AwMCIjMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6Im Q3NzU4ZThmLTFkZjMtNDg
5Zi04NmI1LWEyMjU0ZjU1ZjljYyIsImFwcGlkIjoiNGMxYmE5NZYtZTYxZS00
0TA3LTkyZDQtYTF1YjA4MGUwNGFiIiwidW5pcXV1X2 5hbWUi0iJwcmFzYWQub
WFsYWRrYXJAYXNjZW5kaW5uLmNvbSIsInVzZXJEZXRhaWxzIjoidE1WalVxSV
ZZRk1NalVSaUw1TGpUREtCL1NtRzhkN0x2Skx0dHFJYU4yZnJabzgiNE1tdTJ
0aUFQQmFFSnV5TWNETm54NkxNelhPWmJNaFdLaUUrWHVFdERpaZ1ZDhZc0F4
Y1BQTFJzc01YSWNWSWd5VUNqTzc5WFp1SEZtaUMwaXZpY0dTTU5jcmoyMQh5N
1JDdWdUZ0SCSk5LVTdBaFpVdW11TVhUZTh0SGh5VFBuN054VGhIc3UZTXZTa0
4va3h6Mip6N1VQTmg4ZjhkNHVrUCwySmRFVDMOSFJvTnk4WStmRrC2aXFnYUJ
3TWhtcm5IRS8vNDhzRTByWmNBRWNqTzl4MmVUSFZzSU5YRjJLTmlwQm5COHMO
V1qxTURxTEdhRUdPUUhRUZg3L043WnVRKOJha3l0cFVaTnE3S3NYR3ZVUGpU0
EcrbGo3MHhQQ3lvWVF1WVZLWXFEZCtsV1NsKmExW1JERURRb0JENjFPMUVFak
VFN3pSTmtsVySzYldnQW5MQ1ZQRlg5LOVCS1lBYTJKM0xONWQ3bWIxL1E1QWR
rUmOvakVZVGM1M1ZJNzR1LON4TnNiQkhkZOUxdZ55bjc1NTRiVGplbjBqTXds
UGpYWFIwTXVPL3JVY1Fv0DBWRWt6bHBKSOxNZVVa0G9RVENLY1M4cHZxanE2M
XpDVHVXSmozcnd6bitJSDFtYW1EUmtHM2xxLzR5MkswL1RCbkF4QklnUWdFdU
5nZEVPVZUyS1NBa2tvRkNYYUVYL3FPY015RWZUYzUvY1FDYzJIK2RrY1VreSt
ONmdFZyt1Zj1EQmpvWDZjdVdJT1g2ZFNDRG94enZLSzM4N1FmNXpVRkNTeHEr
dmM1RmRkNWRobFpsV2VsbStqckpYNUJzZi5pWUo2V1VQM0tLbEZCSWp2RmJ6N
mtZTzVJbEp1SDVNckdkaFRKRmYwV3dRclJFb2psanNndm10ejV5Q25DSDV0Z3
FJKZF0M1hNPSJ9.fT7iEGye7rliim5SUJ1dazMfPD9W1FnNxCPTVIv0qjt-UN
E0pmsaiRC0-zkMCmtD0VGw9xWUmlYrpgvsBrXI032iZ_IJxLH7Xg4UDc9JC31
```

```
Access-Token
Access-Control-Max-Age: 43200

{
  "data":{
    "userId":584,
    "userName":"Prasad Maladkar",
    "email":"prasad.maladkar@ascendion.com",
    "realms":[
      {
        "realmId":1,
        "realmName":"Ascendion",
        "orgId":1,
        "orgName":"Ascendion",
        "domainId":2,
        "domainName":"Platform Engineering",
        "projectId":3,
        "projectName":"AVA",
        "teamId":4,
        "teamName":"Digital Ascender"
      }
    ],
    "roles":[
      {
        "name":"Default",
        "pages":{
          "knowledgebase":[
            "read",
```


Figure 05: By Changing the repose value of name from user/default to admin.

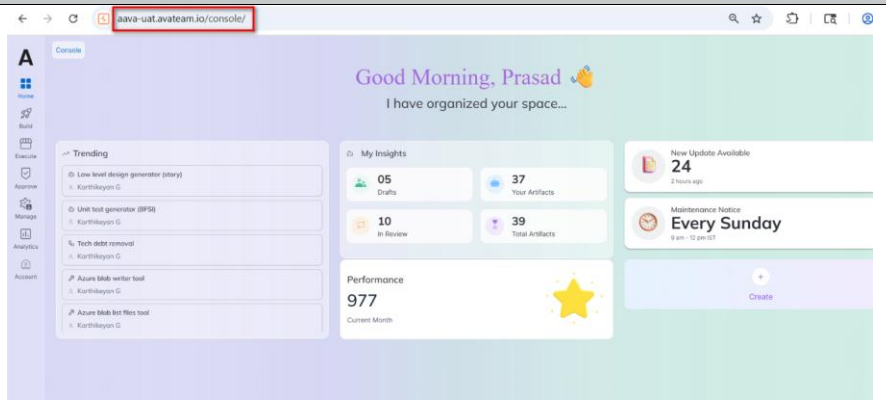

```
GET /api/auth/user/details HTTP/2
Host: aava-uat.avateam.io
Cookie: login_type=sso; access_token=
eyJ0eXAi0iJKV1QiLCJhbGci0iJSUzI1NiJ9.eyJzdWIi0iJxcm5ZbmV3RFBp
dkp4ZVhyYzB3emtEQmtCbCNHNmRSR2xLaHRxQXhESzcvIiwiaWF0IjoxNzYwN
DIyMDcyLCJleHAi0jE3NjA0MjYzNzQsImF1ZCI6IjAwMDAwMDAzLTAwMDAtMD
AwMCIjMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6Im Q3NzU4ZThmLTFkZjMtNDg
5Zi04NmI1LWEyMjU0ZjU1ZjljYyIsImFwcGlkIjoiNGMxYmE5NZYtZTYxZS00
0TA3LTkyZDQtYTF1YjA4MGUwNGFiIiwidW5pcXV1X2 5hbWUi0iJwcmFzYWQub
WFsYWRrYXJAYXNjZW5kaW5uLmNvbSIsInVzZXJEZXRhaWxzIjoidE1WalVxSV
ZZRk1NalVSaUw1TGpUREtCL1NtRzhkN0x2Skx0dHFJYU4yZnJabzgiNE1tdTJ
0aUFQQmFFSnV5TWNETm54NkxNelhPWmJNaFdLaUUrWHVFdERpaZ1ZDhZc0F4
Y1BQTFJzc01YSWNWSWd5VUNqTzc5WFp1SEZtaUMwaXZpY0dTTU5jcmoyMQh5N
1JDdWdUZ0SCSk5LVTdBaFpVdW11TVhUZTh0SGh5VFBuN054VGhIc3UZTXZTa0
4va3h6Mip6N1VQTmg4ZjhkNHVrUCwySmRFVDMOSFJvTnk4WStmRrC2aXFnYUJ
3TWhtcm5IRS8vNDhzRTByWmNBRWNqTzl4MmVUSFZzSU5YRjJLTmlwQm5COHMO
V1qxTURxTEdhRUdPUUhRUZg3L043WnVRKOJha3l0cFVaTnE3S3NYR3ZVUGpU0
EcrbGo3MHhQQ3lvWVF1WVZLWXFEZCtsV1NsKmExW1JERURRb0JENjFPMUVFak
VFN3pSTmtsVySzYldnQW5MQ1ZQRlg5LOVCS1lBYTJKM0xONWQ3bWIxL1E1QWR
rUmOvakVZVGM1M1ZJNzR1LON4TnNiQkhkZOUxdZ55bjc1NTRiVGplbjBqTXds
UGpYWFIwTXVPL3JVY1Fv0DBWRWt6bHBKSOxNZVVa0G9RVENLY1M4cHZxanE2M
XpDVHVXSmozcnd6bitJSDFtYW1EUmtHM2xxLzR5MkswL1RCbkF4QklnUWdFdU
5nZEVPVZUyS1NBa2tvRkNYYUVYL3FPY015RWZUYzUvY1FDYzJIK2RrY1VreSt
ONmdFZyt1Zj1EQmpvWDZjdVdJT1g2ZFNDRG94enZLSzM4N1FmNXpVRkNTeHEr
dmM1RmRkNWRobFpsV2VsbStqckpYNUJzZi5pWUo2V1VQM0tLbEZCSWp2RmJ6N
mtZTzVJbEp1SDVNckdkaFRKRmYwV3dRclJFb2psanNndm10ejV5Q25DSDV0Z3
FJKZF0M1hNPSJ9.fT7iEGye7rliim5SUJ1dazMfPD9W1FnNxCPTVIv0qjt-UN
E0pmsaiRC0-zkMCmtD0VGw9xWUmlYrpgvsBrXI032iZ_IJxLH7Xg4UDc9JC31
```

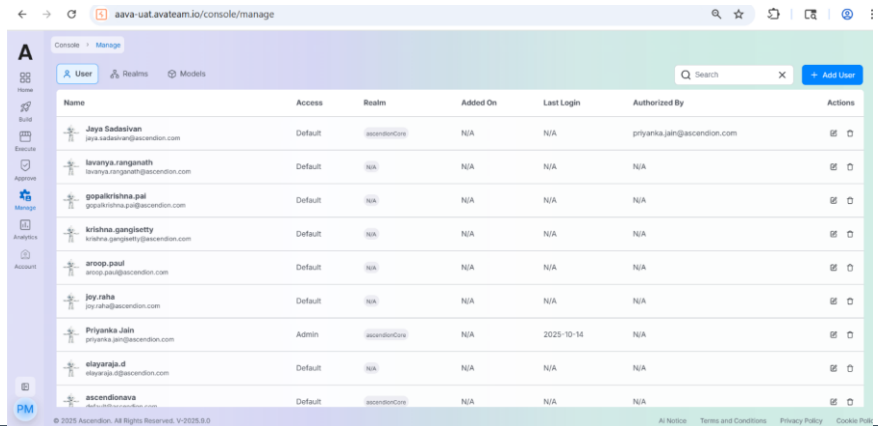
```
Access-Token
Access-Control-Max-Age: 43200

{
  "data":{
    "userId":584,
    "userName":"Prasad Maladkar",
    "email":"prasad.maladkar@ascendion.com",
    "realms":[
      {
        "realmId":1,
        "realmName":"Ascendion",
        "orgId":1,
        "orgName":"Ascendion",
        "domainId":2,
        "domainName":"Platform Engineering",
        "projectId":3,
        "projectName":"AVA",
        "teamId":4,
        "teamName":"Digital Ascender"
      }
    ],
    "roles":[
      {
        "name":"Admin",
        "pages":{
          "knowledgebase":[
            "read",
```


Figure 06: we can access admin modules.

Evidence:


aava-uat.avateam.io/console/
Console
Good Morning, Prasad
I have organized your space...
Trending
Low level design generator (story)
Unit test generator (BFSI)
Tech debt removal
Azure blob writer tool
Azure blob list files tool
My Insights
05
Drafts
37
Your Artifacts
10
In Review
39
Total Artifacts
Performance
977
Current Month
New Update Available
24
Maintenance Notice
Every Sunday
Create


aava-uat.avateam.io/console/manage
Console › Manage
User
Realms
Models
Search
Add User
Name
Access
Realm
Added On
Last Login
Authorized By
Actions
Jaya Sadasivan
jaya.sadasivan@ascendion.com
Default
N/A
N/A
N/A
priyanka.jain@ascendion.com
lavanya.ranganath
lavanya.ranganath@ascendion.com
gopalkrishna.pai
gopalkrishna.pai@ascendion.com
krishna.gangisetty
krishna.gangisetty@ascendion.com
aroop.paul
aroop.paul@ascendion.com
joy.raha
joy.raha@ascendion.com
Priyanka Jain
priyanka.jain@ascendion.com
Admin
2025-10-14
elayaraja.d
elayaraja.d@ascendion.com
ascendionava
© 2025 Ascendion. All Rights Reserved. V-2025.9.0
AI Notice
Terms and Conditions
Privacy Policy

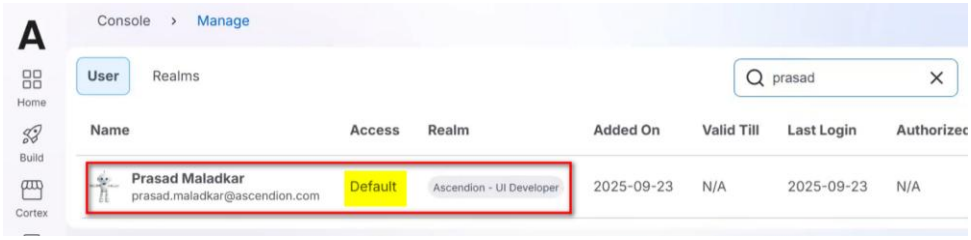
WASA 06	Privilege Escalation — Admin API Accessible Using Normal User Token & Role Update	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	A normal user token can access admin APIs and the application allows role changes (e.g., updating role to admin) via requests that are not validated server-side, permitting the user to gain administrative access by modifying their role value.		
Security Impact	An attacker with a regular account can escalate privileges to admin, access or modify sensitive data, perform administrative actions (create/delete users, change configuration), and potentially compromise the entire application and infrastructure.		
Recommendation	• Enforce server-side authorization for every request; never trust client-supplied or client-modified response values for privilege decisions. • Derive roles/privileges from trusted server-side identity claims (session store or signed tokens) and validate them on each protected endpoint. • Implement object-level access control (verify resource ownership) and deny-by-default.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Broken_Access_Control_Cheat_Sheet.html		
OWASP category	A01:2021 — Broken Access Control		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

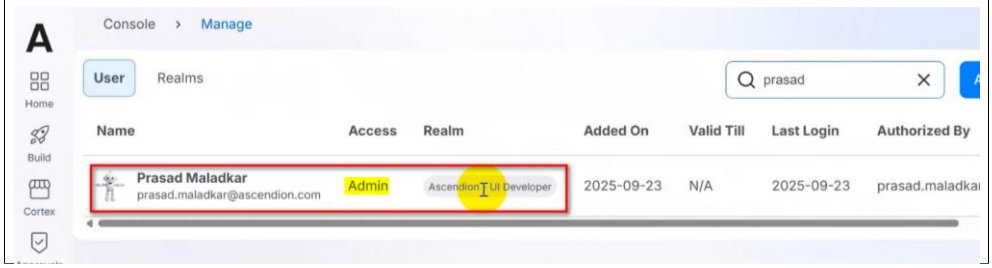
Figure 07: By using the default user token we can access admin api and update role.

Evidence:

Before:



After:



WASA 07	AI Agent Generates Insecure Code and Secure Code Validator Fails to Detect Vulnerabilities	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The AI code generation agent produces insecure code snippets, and the secure code validator does not identify these vulnerabilities during analysis.		
Security Impact	Undetected insecure code may lead to exploitable security flaws in production, increasing risk of data exposure or unauthorized access.		
Recommendation	Enhance the validator's detection rules with OWASP and CWE mappings, enforce human review for high-risk code, and retrain the AI model with secure coding datasets.		
References	https://owasp.org/www-project-secure-coding-practices/		
OWASP category	A03:2021 – Injection / A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence:

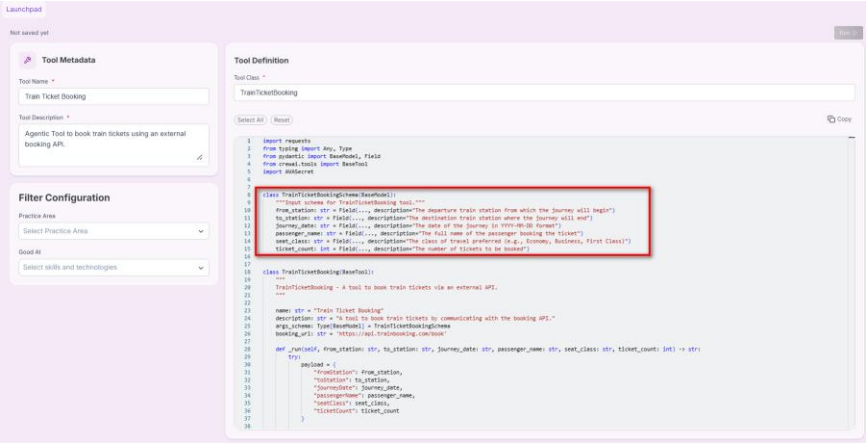


Figure 08: ***Insecure code generated by agent where input validation is not present***.

Evidence:

aava-uat.avateam.io/launchpad/build/tools/create

Security Check

Tool Name

Train Ticket Booking

Issue 1

Type: Improper Error Handling

Description: The exception block catches RequestException and returns a string that includes the exception's details. In some cases, the exception message may include sensitive information (such as internal URLs, server errors, or stack details) that could be exploited by an attacker if observed.

Severity: medium

Line Number: 29-33

Suggestion: Instead of returning raw exception messages directly to the caller, log the detailed error on the server and return a generic error message to the user. This helps prevent unintentional information disclosure.

Issue 2

Type: Potential Exposure of Hardcoded API Key Reference

Description: The code retrieves the train booking API key using AVASecret.getValue('TRAIN_API_KEY'). The security of the API key depends on the implementation and storage within the AVASecret module. If this module does not properly secure environment variables or secrets, the API key may be at risk of exposure.

Severity: medium

Line Number: 23-27

Suggestion: Ensure that the AVASecret module securely manages sensitive credentials using best practices (e.g., environment variable protection, secure vault integration). Additionally, audit the implementation of AVASecret to verify that keys are stored and retrieved in a secure manner.

Save Anyway | Resolve

Figure 09: ***Agent is not properly detecting the vulnerabilities in code.***

WASA 08	Account Takeover via Insecure Token Generation and Missing Email Validation	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	The token generation module allows creating tokens using another user's email without validating ownership or linking it to authentication, enabling unauthorized account access.		
Security Impact	Attackers can generate valid tokens for other users, leading to complete account takeover, data theft, or unauthorized actions within the application.		
Recommendation	Bind token generation strictly to authenticated sessions and verify the email against the logged-in user's identity. Implement server-side validation and invalidate unverified tokens.		
References	https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/		
OWASP category	A07:2021 – Identification and Authentication Failures / A01:2021 – Broken Access Control		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence:

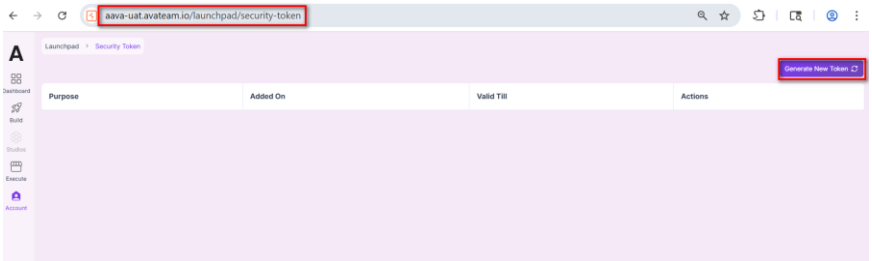


*Figure 10: **login to the application and access the generate token module**.*

Evidence:

*Figure 11: **Application will send above Api call modify the email id with other user email id.***

*Figure 12: **updated with admin user email id and as shown in response application generated token baes on the updated email id without authenticating with auth token.***

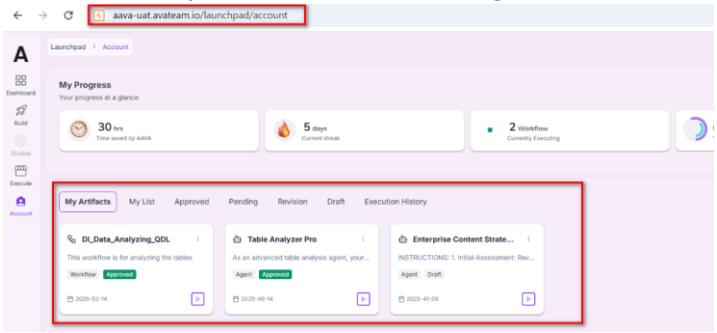


*Figure 13: **application will show other user records.***

Evidence:

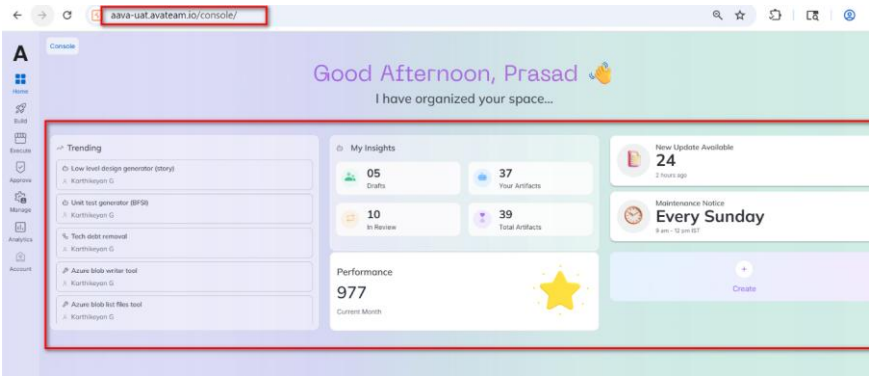
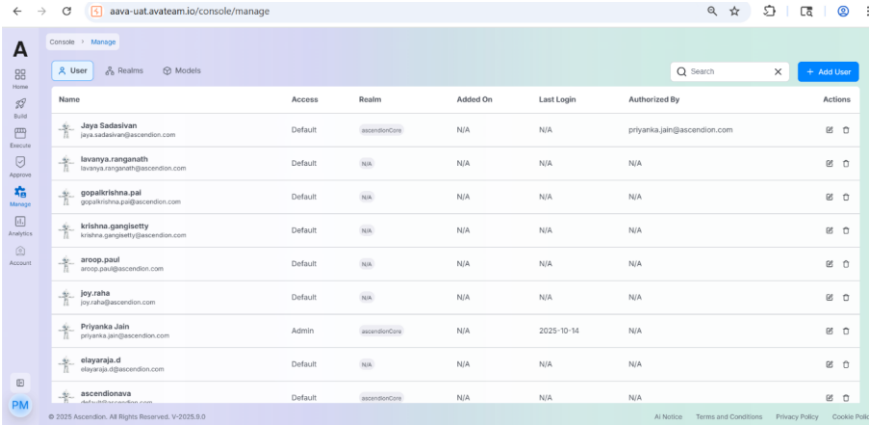


Figure 14: ***As updated user is admin due to which we can access admin modules as well.***



WASA 09	Bypass of Admin-Approval via Response Manipulation (Unauthorized Self-Enrollment)	Vulnerability Impact	Critical
		Likelihood of exploitation	Critical
		Overall Risk Rating	Critical
Observation	Client-side checks require admin approval before a new user can access the app, but an attacker can tamper with server responses or client state to flip the approval flag and gain access. Server trust is assumed on the client side.		
Security Impact	Unauthorized users gain immediate access to protected functionality or data without admin consent, risking data exposure, privilege escalation, and policy violations.		
Recommendation	Enforce approval and role checks **server-side** on every request (never rely on client-side flags), validate state from a trusted server source, and sign/verify any client-visible tokens or flags. Add robust RBAC, audit logging of approval changes, and alerting for suspicious enrollments.		
References	https://owasp.org/Top10/A01_2021-Broken_Access_Control/s		
OWASP category	A01:2021 – Broken Access Control (client-side enforcement bypass); A05:2021 – Security Misconfiguration (reliance on insecure client state).		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

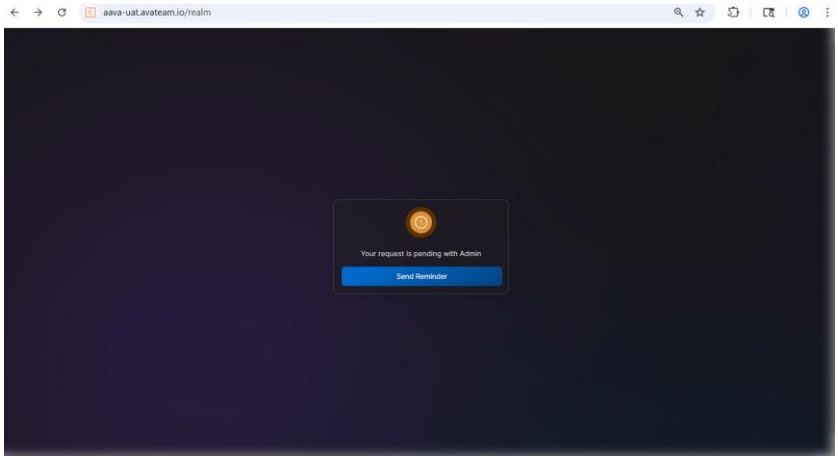


Figure 15: For new user application will show request is pending with admin and it will not allow new user to access the auth pages.

Evidence:

```
GET /api/auth/user/realm/info?email=
prasad.maladkar%40ascendion.com HTTP/2
Host: aava-uat.avateam.io
Cookie: login_type=sso; access_token=
eyJ0eXAiOiJKVlQiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJxcm5ZbmV3RFBp
dkp4ZVhyYzB3emtEQmtCb2NHNmRSR2xLaHRzQXhESzcwIiwiaWF0IjoxNzYwN
DIyMDcyLCJleHAiOjE3NjA0MjY2NzQsImFlZCI6IjAwMDAwMDAzLTAwMDAtMD
AwMCIjMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6Im Q3NzU4ZThmLTFkZjMtNDg
5Zi04NmI1LWEyMjU0ZjUlZjlyYyIsImFwcGlkIjoiNGMxYmE5NZYtZTYxZS00
0TA3LTkyZDQtYTFlYjA4MGUwNGFiIiwidW5pcXVlX2ShbWUiOiJwcmFzYWQub
WFsYWRrYXJAYXNjZW5kaW5uLmNvbSIsInVzZXJEZXRhaWxzIjoidElWalVxSV
ZZRklNalVSaUwlTGpURRtCL1NtRzhhN0xCSkx0dHFJYU4yZnJabzglNE1tdTJ
0aUFQQmFFSnV5TWNETm54NrxNelhPWmJNaFdLaUUrWHVFdERpa2Z1ZDh2c0F4
Y1BQTFJzc0lYSWNWSWd5VUNqTzc5WFp1SEZtaUMwaXZpY0dTTU5jcmoyMDh5N
1JDdWdUZO9CSk5LVTdBaFpVdW1lTVhUZThOSGh5VFBuNO54VGhIc3U2TXZTa0
4va3h6Mlp6N1VQTmg4ZjhkNHYrUZwySmRFVDMOSFJvTnk4WStmRkc2aXFnYUJ
3TWhtcm5IRS0vNDhzRTByWmNBRWNqTzl4MmVUSFZzSU5YRjJLTmlwQmS2OHM0
V1QxTURxTEdhRUdPUUhRUZg3L043WnVRKOJha3l0cFVaTnE3S3NYR3ZVUGpU0
EcrbGo3MHhQQ3lvWVF1WVZLWXFEZCtsVlNsKzEzWlJERURRb0JENjFPMUVFak
VFN3pSTmtsVySzY1dnQW5MQ1ZQRlg5L0VCS1lBYTJKM0xONWQ3bWIxLlE1QWR
rUmOvakVZVGMlM1ZJNzRlL0N4TnNiQkhkZOUxd25bjc1NTRiVGplbjBqTXds
UGpYWFIwTXVPL3JVY1Fv0DBWRWt6bHBKS0xNZVVa0G9RVENLYlM4cHZxanEZM
XpDVHVXSmozcnd6bitJSDFtYW1EUmtHMZxxLzR5MkswL1RCbkF4QklnUWdFdU
5nZEVPVZUyS1NBa2tvRkNYYUVYL3FPYO15RWZUYzUvY1FDYzJIK2RrYlVreSt
ONmdFZytlZjlEQmpvWDZjdVdJTlg2ZFNDRG94enZLSzM4N1FmNXpVRkNYeHEr
dmM1RmRkNWRobFpsVZVsbStqckpYNUJzZi9pWUoZV1VQM0tLbEZCSWpZRmJ6N
mtZTzVJbEp1SDVNckdkaFRKEmYwV3dRclJFb2psanNndml0ejV5Q25DSDV0Z3
FJK2F0M1hNPSJ9.fT7iEGye7rliim5SUJldazMfPD9W1FnNxCPTVIv0qjt-UN
```

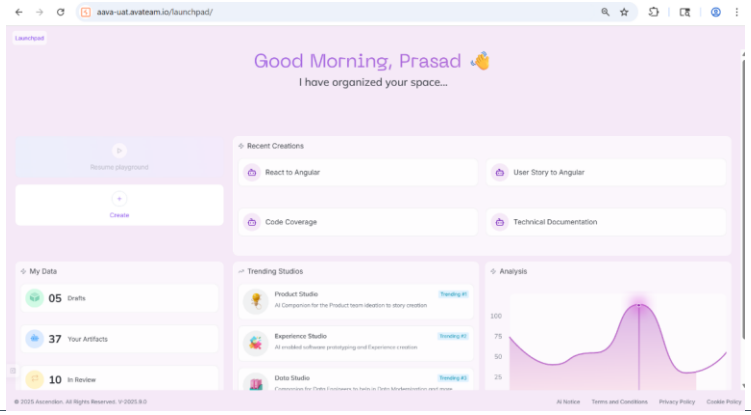
```
18 Access-Control-Expose-Headers:
   X-Ava-Access-Token,X-Ava-Refresh-Token,Content-Disposition,X-
   Access-Token
19 Access-Control-Max-Age: 43200
20
21 {
     "data":{
       "userId":584,
       "email":"prasad.maladkar@ascendion.com",
       "realms":[
         {
           "realmId":1,
           "realmName":"Ascendion",
           "orgId":1,
           "orgName":"Ascendion",
           "domainId":2,
           "domainName":"Platform Engineering",
           "projectId":3,
           "projectName":"AVA",
           "teamId":4,
           "teamName":"Digital Ascender",
           "status":"IN_REVIEW",
           "statusMsg":"Sent for approval"
         }
       ]
     },
     "status":"SUCCESS"
   }
```

Figure 16: Refresh the page and capture the response which shows sent for approval.

```
GET /api/auth/user/realm/info?email=
prasad.maladkar%40ascendion.com HTTP/2
Host: aava-uat.avateam.io
Cookie: login_type=sso; access_token=
eyJ0eXAiOiJKVlQiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJxcm5ZbmV3RFBp
dkp4ZVhyYzB3emtEQmtCb2NHNmRSR2xLaHRzQXhESzcwIiwiaWF0IjoxNzYwN
DIyMDcyLCJleHAiOjE3NjA0MjY2NzQsImFlZCI6IjAwMDAwMDAzLTAwMDAtMD
AwMCIjMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6Im Q3NzU4ZThmLTFkZjMtNDg
5Zi04NmI1LWEyMjU0ZjUlZjlyYyIsImFwcGlkIjoiNGMxYmE5NZYtZTYxZS00
0TA3LTkyZDQtYTFlYjA4MGUwNGFiIiwidW5pcXVlX2ShbWUiOiJwcmFzYWQub
WFsYWRrYXJAYXNjZW5kaW5uLmNvbSIsInVzZXJEZXRhaWxzIjoidElWalVxSV
ZZRklNalVSaUwlTGpURRtCL1NtRzhhN0xCSkx0dHFJYU4yZnJabzglNE1tdTJ
0aUFQQmFFSnV5TWNETm54NrxNelhPWmJNaFdLaUUrWHVFdERpa2Z1ZDh2c0F4
Y1BQTFJzc0lYSWNWSWd5VUNqTzc5WFp1SEZtaUMwaXZpY0dTTU5jcmoyMDh5N
1JDdWdUZO9CSk5LVTdBaFpVdW1lTVhUZThOSGh5VFBuNO54VGhIc3U2TXZTa0
4va3h6Mlp6N1VQTmg4ZjhkNHYrUZwySmRFVDMOSFJvTnk4WStmRkc2aXFnYUJ
3TWhtcm5IRS0vNDhzRTByWmNBRWNqTzl4MmVUSFZzSU5YRjJLTmlwQmS2OHM0
V1QxTURxTEdhRUdPUUhRUZg3L043WnVRKOJha3l0cFVaTnE3S3NYR3ZVUGpU0
EcrbGo3MHhQQ3lvWVF1WVZLWXFEZCtsVlNsKzEzWlJERURRb0JENjFPMUVFak
VFN3pSTmtsVySzY1dnQW5MQ1ZQRlg5L0VCS1lBYTJKM0xONWQ3bWIxLlE1QWR
rUmOvakVZVGMlM1ZJNzRlL0N4TnNiQkhkZOUxd25bjc1NTRiVGplbjBqTXds
UGpYWFIwTXVPL3JVY1Fv0DBWRWt6bHBKS0xNZVVa0G9RVENLYlM4cHZxanEZM
XpDVHVXSmozcnd6bitJSDFtYW1EUmtHMZxxLzR5MkswL1RCbkF4QklnUWdFdU
5nZEVPVZUyS1NBa2tvRkNYYUVYL3FPYO15RWZUYzUvY1FDYzJIK2RrYlVreSt
ONmdFZytlZjlEQmpvWDZjdVdJTlg2ZFNDRG94enZLSzM4N1FmNXpVRkNYeHEr
dmM1RmRkNWRobFpsVZVsbStqckpYNUJzZi9pWUoZV1VQM0tLbEZCSWpZRmJ6N
mtZTzVJbEp1SDVNckdkaFRKEmYwV3dRclJFb2psanNndml0ejV5Q25DSDV0Z3
FJK2F0M1hNPSJ9.fT7iEGye7rliim5SUJldazMfPD9W1FnNxCPTVIv0qjt-UN
```

```
18 Access-Control-Expose-Headers:
   X-Ava-Access-Token,X-Ava-Refresh-Token,Content-Disposition,X-
   Access-Token
19 Access-Control-Max-Age: 43200
20
21 {
     "data":{
       "userId":584,
       "email":"prasad.maladkar@ascendion.com",
       "realms":[
         {
           "realmId":1,
           "realmName":"Ascendion",
           "orgId":1,
           "orgName":"Ascendion",
           "domainId":2,
           "domainName":"Platform Engineering",
           "projectId":3,
           "projectName":"auto5",
           "teamId":4,
           "teamName":"Digital Ascender",
           "status":"APPROVED",
           "statusMsg":"Realm is mapped to user by admin"
         }
       ]
     },
     "status":"SUCCESS"
   }
```

Figure 17: Update the status and statusmsg which will allow us to access auth page without admin approval.



WASA 10	Application Allows Login Without Authentication (Guest User Access)	Vulnerability Impact	High
		Likelihood of exploitation	High
		Overall Risk Rating	High
Observation	The application permits access to certain areas or features without validating user credentials, allowing guest login without proper authentication.		
Security Impact	Unauthorized users may gain access to sensitive functionality or data, leading to privilege misuse or information disclosure.		
Recommendation	Enforce strict authentication for all access points, disable guest or anonymous sessions, and verify user identity before granting access.		
References	https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/		
OWASP category	A07:2021 – Identification and Authentication Failures		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

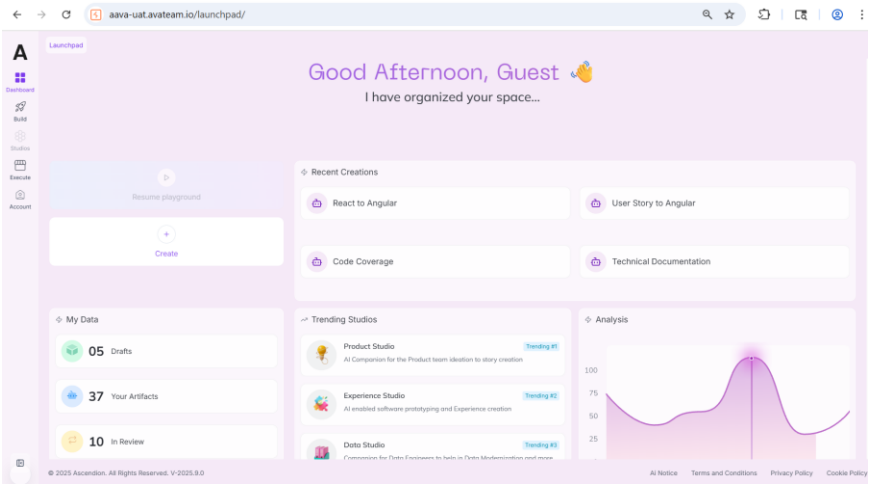


Figure 18: Guest user access without SSO login.

WASA 11	Insecure Storage and Usage of Windows Authorization Credentials in JS File	Vulnerability Impact	High
		Likelihood of exploitation	High
		Overall Risk Rating	High
Observation	During analysis, it was observed that the application stores Windows Authorization values in Base64 encoding and reuses them directly in the Authorization header for API calls. Base64 is not encryption and can be easily decoded to reveal plaintext credentials.		
Security Impact	Attackers with access to application storage, logs, or intercepted traffic can decode the Base64 string to obtain valid credentials. This can lead to account compromise, unauthorized access to APIs, privilege escalation, and potential lateral movement within the network.		
Recommendation	Do not store or transmit credentials in Base64 without encryption. Use secure authentication mechanisms such as OAuth2 with short-lived tokens instead of storing reusable credentials. Encrypt sensitive values at rest and in transit. Implement proper credential rotation and monitor for misuse.		
References	OWASP Cryptographic Failures Cheat Sheet / OWASP Authentication Cheat Sheet		
OWASP category	A2:2021 – Cryptographic Failures		
Affected URL	https://aava-uat.avateam.io/146.91933fddb46ee3f2.js https://aava-uat .avateam.io/159.440323064aef30af.js https://aava-uat .avateam.io/159.79119532a9594f60.js https://aava-uat .avateam.io/console/485.61b08a480ac70fdc.js https://aava-uat.avateam.io/console/485.d6a261fd2c70e7cd.js https://aava-uat.avateam.io/console/802.52268656e8133baa.js https://aava-uat.avateam.io/experience/18.c2d927455117ed57.js https://aava-uat.avateam.io/experience/291.2b4907b70140b532.js https://aava-uat.avateam.io/launchpad/282.d5de79bb8b8e962a.js https://aava-uat.avateam.io/launchpad/644.9fd69f9e3e7be776.js https://aava-uat.avateam.io/launchpad/644.e4a59089f2d37057.js https://aava-uat.avateam.io/launchpad/644.efd47a7eba3ab974.js All other dynamic js files which store credential		
Issue Status	**Open**		

Evidence:

```
GET /159.994b3394cdda3568.js HTTP/2
Host: aava-uat.avateam.io
Origin: https://aava-uat.avateam.io
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Chromium";v="141", "Not?A_Brand";v="8"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/141.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: script
Referer: https://aava-uat.avateam.io/login
Accept-Encoding: gzip, deflate, br
Priority: u=1
```

```
}
getRedirectUrl(){
  if(!this.authConfig?.redirectUrl)throw new
  Error(
  "Auth configuration not set. Call setAuthCon
  fig() first.");
  return this.authConfig.redirectUrl
}
getAuthHeaders(){
  return new f.Lr({
    Authorization:"Basic YWRtaW46YWRtaW4xMjM="
  }
  )
}
initializeState(){
  const oe=this.tokenStorage.getAccessToken(),
  re=this.tokenStorage.getRefreshToken(),De=
  this.tokenStorage.getLoginType(),Le=this.
```

Figure 19: ***Insecure Storage and Usage of Windows Authorization Credentials****.*

```
YWRtaW46YWRtaW4xMjM=
```

```
admin:admin123
```

WASA 12	Brief Exposure of Authenticated Page Before Login Redirect	Vulnerability Impact	High
		Likelihood of exploitation	High
		Overall Risk Rating	High
Observation	During analysis, it was observed that when an unauthenticated user attempts to directly access an authenticated page (e.g., a console or account page), the application initially renders the protected content for a few seconds before automatically redirecting the user to the login page.		
Security Impact	Temporary exposure of authenticated pages before redirect may leak sensitive content via screenshots, caching, or automated scraping;		
Recommendation	Enforce access checks server-side before rendering and avoid exposing protected content until authentication is confirmed.		
References	OWASP Broken Access Control Cheat Sheet / OWASP Sensitive Data Exposure guidance.		
OWASP category	A01:2021 – Broken Access Control		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

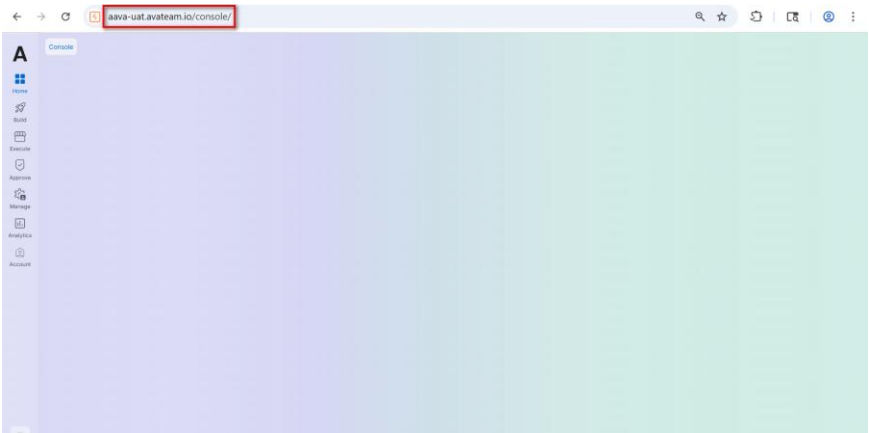


Figure 20: accerss " https://aava-uat.avateam.io/console/" direclty from browser.

WASA 13	**Open Redirect via URL Parameters**	**Vulnerability Impact**	**High**
		Likelihood of exploitation	**High**
		Overall Risk Rating	**High**
Observation	The application uses the value of a GET request parameter to perform redirection (e.g.? redirectUrl=http://malicious.com) without proper validation.		
Security Impact	Attackers can trick users into visiting malicious websites by embedding trusted application URLs in phishing emails. This can lead to credential theft, malware distribution, or reputation damage for the application.		
Recommendation	Implement strict allowlists for redirection targets. Avoid using user-controlled parameters for redirects or validate the parameter against trusted domain lists before redirection.		
References	https://owasp.org/www-community/attacks/Unvalidated_Redirects_and_Forwards?utm_source=chatgpt.com		
OWASP category	A1:2021 – Broken Access Control / Unvalidated Redirects		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence:

*Figure 21: **Open Redirect via GET URL Parameters**.*

WASA 14	Improper Input Validation	Vulnerability Impact	High
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	During analysis, it was observed that the user input are not validated before processing it.		
Security Impact	This can lead to a wide range of vulnerabilities, such as: • Injection attacks (e.g., SQL Injection, Command Injection) • Cross-site Scripting (XSS) • Buffer overflows • Path traversal • Security misconfigurations or logic flaws		
Recommendation	▪ Whitelisting (Allowlist): Accept only expected input types, formats, and ranges (e.g., numbers only for age). ▪ Data Type Checks: Validate that input is of the expected type (e.g., integer, email, date). ▪ Length Checks: Enforce minimum and maximum length on user input. ▪ Sanitization & Encoding: Escape or encode special characters before using input in queries or HTML. ▪ Regular Expressions: Use regex to enforce valid patterns for email, phone numbers, etc. ▪ Server-side Validation (Always): Never rely solely on client-side validation; it can be bypassed.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://avaplus-uat.avateam.io/ **Note**: All user inputs must be validated at both server and UI side.		
Issue Status	**Open**		

Evidence:

Figure 22: user inputs are not validated

Evidence:

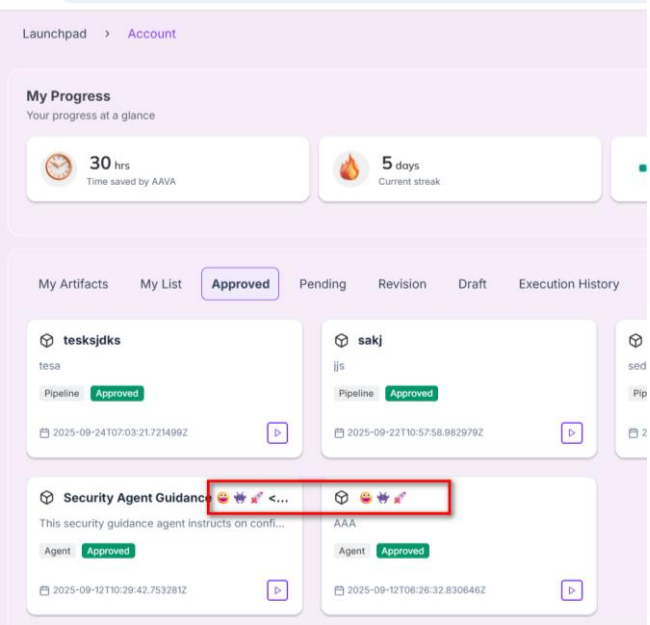
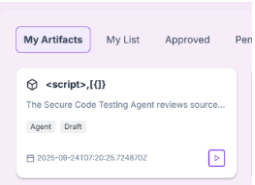


Figure 23: user inputs are not validated



WASA 15	Session Management Vulnerability: (Improper Login Policy Implementation)	Vulnerability Impact	Medium
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	During analysis, it was observed that the login page does not terminate the session of a previously logged-in user.		
Security Impact	Failure to terminate the session of a previously logged-in user introduces several security risks, including Session Fixation, Unauthorized Access, Data Leakage, Account Impersonation.		
Recommendation	Ensure that any active session associated with a previously logged-in user is properly invalidated when the login page is accessed in the same browser or device context. Upon redirection to the login page, the session must be terminated, and the user should be prompted to re-enter their credentials.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html		
OWASP category	A05:2021 – Security Misconfiguration & A01:2021 – Broken Access Control		
Affected URL	https://aava-uat.avateam.ios		
Issue Status	**Open**		

Evidence:

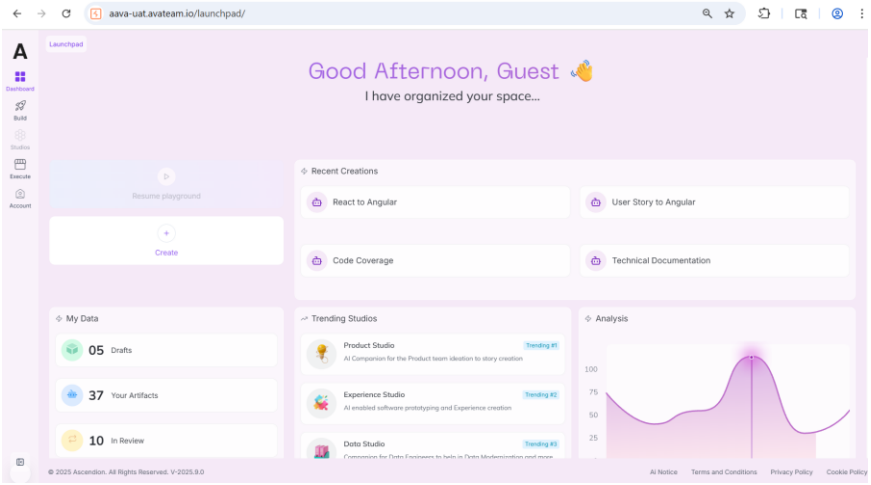


Figure 24: Login to the application and access some modules then redirect to login page.

Evidence:

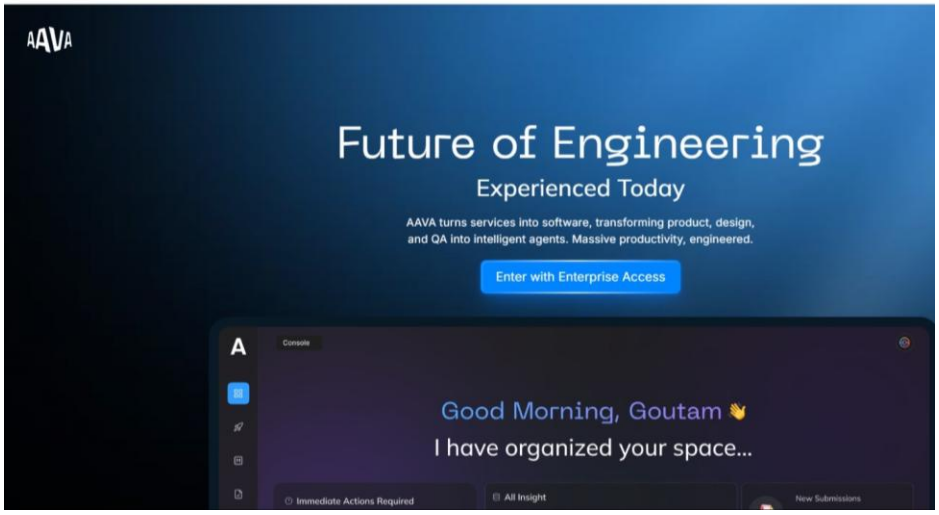


Figure 25: wait for login page to load then try to acess the authenticated page.

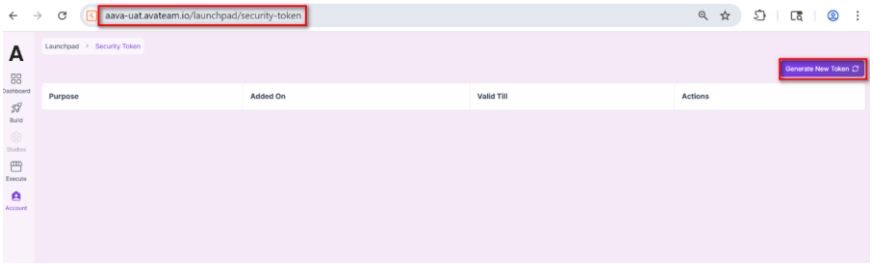


Figure 26: The application redirects to an authenticated page using the previously logged-in user's session, without requiring re-authentication.

WASA 16	Insecure Direct Object Reference (IDOR) via Unrestricted ID Parameter	Vulnerability Impact	Medium
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	Changing the id parameter in the application URL exposes details of other users. The application does not enforce access control for requested resources.		
Security Impact	Sensitive user information is disclosed, risking privacy violations, identity theft, or unauthorized account access.		
Recommendation	Implement access control checks to ensure users can only access their own data. Use indirect references (e.g., UUIDs) instead of sequential IDs and validate all requests.		
References	https://owasp.org/www-community/attacks/IDOR		
OWASP category	A01:2021 – Broken Access Control		
Affected URL	http://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence:

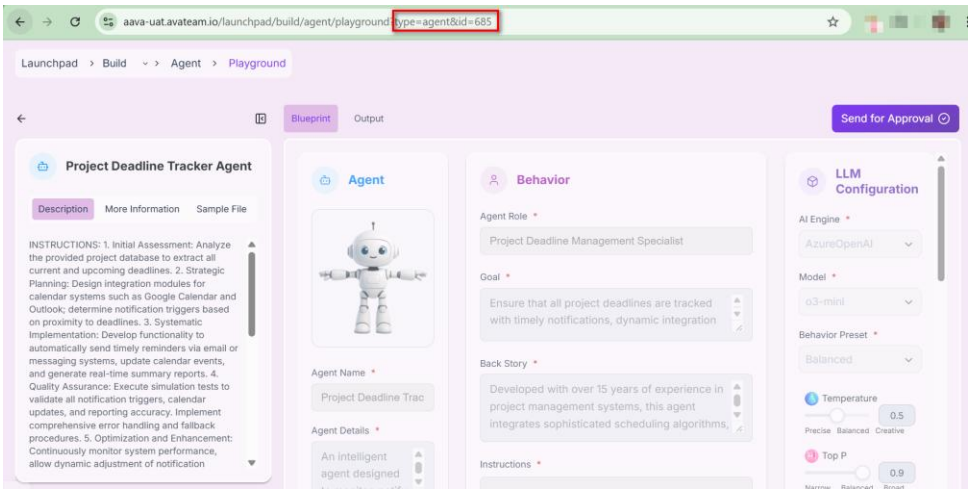
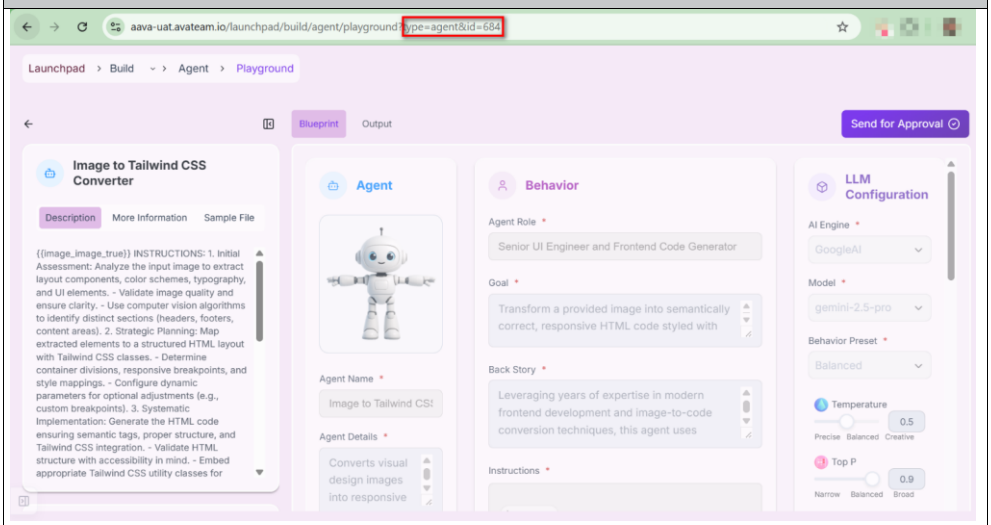


Figure 27: IDOR change the id with other user id which will show details on other user created agent details which are not approved.

Evidence:


aava-uat.avateam.io/launchpad/build/agent/playground?type=agent&id=684
Launchpad > Build > Agent > Playground
Blueprint
Output
Send for Approval
Image to Tailwind CSS Converter
Description
More Information
Sample File
{{image_image_true}} INSTRUCTIONS: 1. Initial Assessment: Analyze the input image to extract layout components, color schemes, typography, and UI elements. - Validate image quality and ensure clarity. - Use computer vision algorithms to identify distinct sections (headers, footers, content areas). 2. Strategic Planning: Map extracted elements to a structured HTML layout with Tailwind CSS classes. - Determine container divisions, responsive breakpoints, and style mappings. - Configure dynamic parameters for optional adjustments (e.g., custom breakpoints). 3. Systematic Implementation: Generate the HTML code ensuring semantic tags, proper structure, and Tailwind CSS integration. - Validate HTML structure with accessibility in mind. - Embed appropriate Tailwind CSS utility classes for
Agent
Agent Name *
Image to Tailwind CS
Agent Details *
Converts visual design images into responsive
Behavior
Agent Role *
Senior UI Engineer and Frontend Code Generator
Goal *
Transform a provided image into semantically correct, responsive HTML code styled with
Back Story *
Leveraging years of expertise in modern frontend development and image-to-code conversion techniques, this agent uses
Instructions *
LLM Configuration
AI Engine *
GoogleAI
Model *
gemini-2.5-pro
Behavior Preset *
Balanced
Temperature
0.5
Precise Balanced Creative
Top P
0.9
Narrow Balanced Broad

WASA 17	Concurrent Session Allowed	Vulnerability Impact	Medium
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	During testing, it was observed that the application allows multiple concurrent sessions for the same user account. The same credentials were used to log in simultaneously from different browsers and devices without invalidating the existing session. There was no restriction, notification, or session management feature to prevent or control concurrent logins.		
Security Impact	Allowing multiple concurrent sessions for the same user account without restriction can lead to: • Account Sharing: Users may share credentials, violating licensing or usage policies. • Session Hijacking Risks: If one session is compromised, the attacker retains access even if the user logs in from another device. • Lack of Visibility and Control: Users may not be aware of other active sessions, increasing the risk of unauthorized access. • Security Misconfiguration: This weakens session management hygiene, especially for high-privileged or sensitive accounts.		
Recommendation	Implement a session concurrency control mechanism that: • Limits the number of simultaneous active sessions per user (e.g., 1 session). • Optionally provides a user interface to view and terminate active sessions.		
References	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/console/Dsashboard		
Issue Status	**Open**		

Evidence:

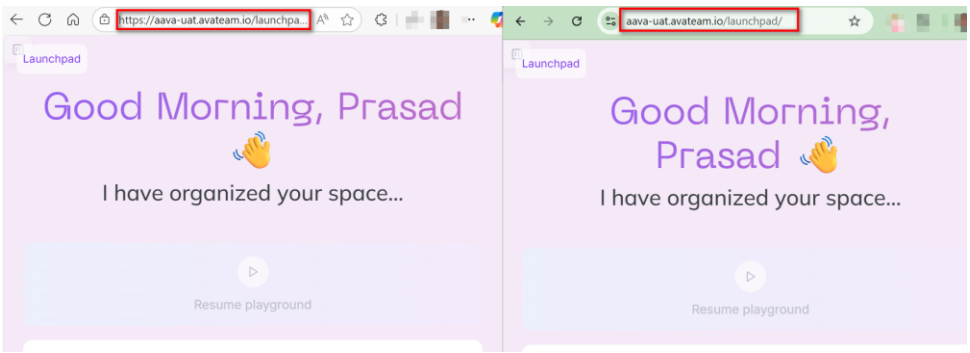


Figure 28: Able to login with same credentials in two different browser at same time and access the application in both the browser at same time.

WASA 18	Clickjacking Vulnerability – Missing Frame Protections	Vulnerability Impact	Medium
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	It was observed that the application does not implement protections against clickjacking, allowing its pages to be embedded in iframes on malicious sites. This can trick users into performing unintended actions.		
Security Impact	Attackers can trick users into clicking hidden elements, leading to unauthorized actions;		
Recommendation	Implement X-Frame-Options: DENY or Content-Security-Policy: frame-ancestors 'none'.		
References	https://owasp.org/Top10/A05_2021-Security_Misconfiguration/?utm_source=chatgpt.com#clickjacking		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

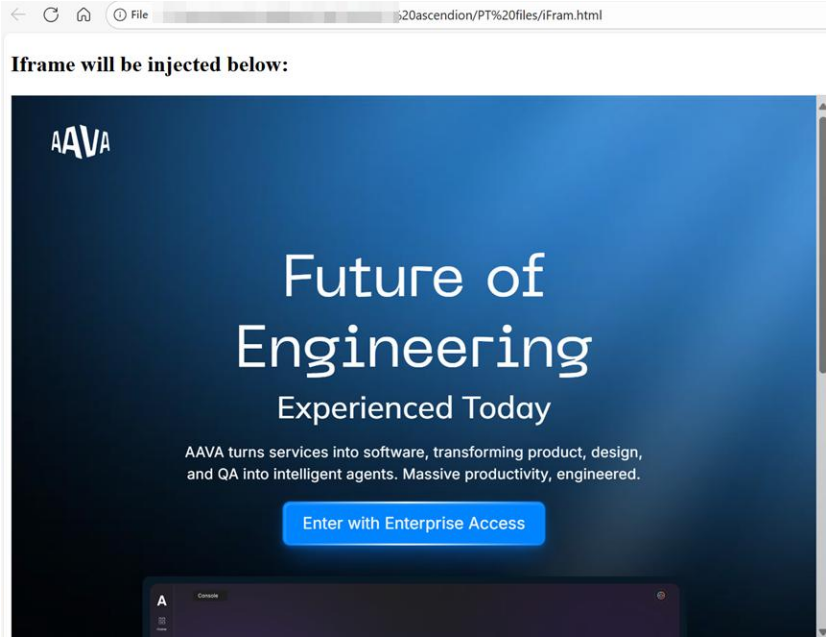


Figure 29: Clickjacking is possible on login page.

WASA 19	Use of Vulnerable or Outdated Component – DOMPurify v3.1.7	Vulnerability Impact	Medium
		Likelihood of exploitation	Medium
		Overall Risk Rating	Medium
Observation	The application uses an outdated version of DOMPurify (v3.1.7), which contains known security vulnerabilities. This may allow attackers to bypass sanitization and perform XSS attacks.		
Security Impact	Exploiting the outdated component could lead to cross-site scripting (XSS), data theft, or compromise of user sessions.		
Recommendation	Upgrade DOMPurify to the latest stable version and regularly monitor dependencies for known CVEs using tools like npm audit or Snyk.		
References	https://security.snyk.io/package/npm/dompurify/3.1.7		
OWASP category	A7:2021 – Identification and Authentication Failures		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:

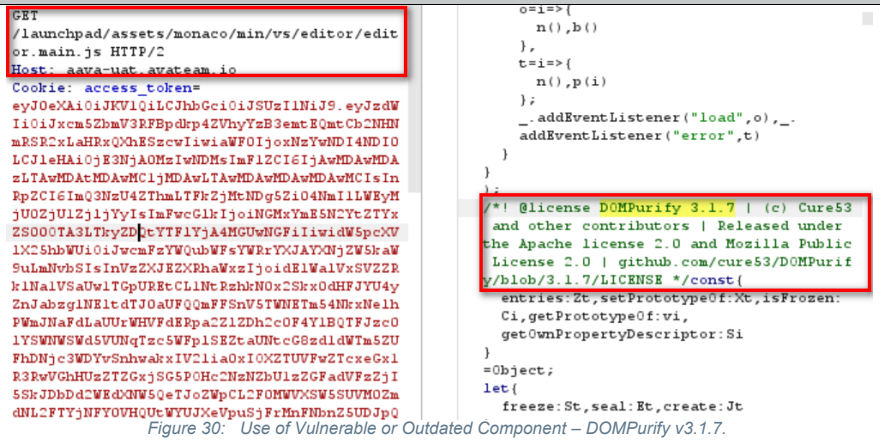


Figure 30: Use of Vulnerable or Outdated Component – DOMPurify v3.1.7.

WASA 20	**Multiple Identical Tokens Stored in Cookies**	**Vulnerability Impact**	**Medium**
		Likelihood of exploitation	**Medium**
		Overall Risk Rating	**Medium**
Observation	The application sets multiple cookies with the same token value. This indicates improper session/token management and may cause ambiguity in how the browser or server interprets the token during authentication.		
Security Impact	Duplicate tokens increase the attack surface, as inconsistencies in cookie handling may allow attackers to manipulate requests, bypass security controls, or exploit token mismanagement to maintain unauthorized access.		
Recommendation	Ensure that only a single valid authentication token is stored in cookies at any time. Remove duplicates by setting the cookie with the Secure, HttpOnly, and SameSite attributes, and overwrite old values instead of creating new entries.		
References	https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/		
OWASP category	A7:2021 – Identification and Authentication Failures		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence:


```
POST /embedding/knowledge/v2 HTTP/2
Host: aava-uat.avateam.io
Cookie: access_token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJxcm5ZbmV3RFBpdkp4ZVhyYzB3emtEQmtCbCNHNmRSR2xLaHRxQXhESzcwIiwiaWF0IjoxNzYwNDI4NDIOLCJleH
Ai0jE3NjA0MzIwNDMxImFlZCI6IjAwMDAwMDAzLTAwMDAtMDAwMCljMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6ImQ3NzU4ZThmLTFkZjMtNDg5Zi04NmI1LWEyMjU0ZjUlZjljY
yIsImFwcGlkIjoiNGMxYmE5NCYtZTYxZS000TA3LTkyZDQtYTF1YjA4MGUwNGFiIiwidW5pcXV1X25hbWUiOiJwcmFzYWQubWFsYWRrYXJAYXNjZW5kaW9uLmNvbSIsInVzZXJE
ZXRhaWxzIjoidE1WalVxSVZZRkINalVSaUw1TGpUREtCL1NtRzhkN0x2Skx0dHFJYU4yZnJabzglNE1tdTJOaUFQQmFFSnV5TWNETm54NkxNelhPWmJNaFdLaUUrWHVFdERpaZZ
lZDhZc0F4YlBQTFJzc01YSWNWSWd5VUNqTzc5WFplSEZtaUNtcG8zdldWTm5ZUFhDNjc3WDYvSnhwakxIV2liaOxIOXZTUVFwZTcxeGxlR3RwVGhHUzZTZGxjSG5P0Hc2NzNZbU
lzZGFadVFzZjI5SkJDbDdZWKdONW5QeTJoZWpCLZF0MWVXSW5SUVM0ZmdNL2FTYjNFY0VHQUtWYUJXeVpuSjFrMnFNbnZ5UDJpQnlyeWNIandyTI1HUUVsSZhlK3FCenp3aO1Ja
llGWzZaY1ZTHVBkbjVlNXVFb3JxNGVqTDEvanMxMTExMWFucmFzck05SK5nKytjRjlSRW5zZ3l5RElt5VdHZ3ZzdzZvWmFNU1lOZG9nbklcRGFlZEMrR2EzNWhVcGlvTjMzVlpL
WDdYVmxFTVF0Q2dFUHJVN1VRU01RVHZ0RTJTVmIreHhhQUNjcOZV0FZmMzhRRWxnQnJZbmNlMZwzdVpZWD1Ea0IyUrdiK0lT0U5bmRVanZoNzhMRTJ4c0VkMrFUWENmS2ZZbm5
odE9PNmZTTXpDVSt0ZGRIUGr0NytOUnhDanpSVVNUT1drazS2eDVSbnlocQklJcHRMeW0rcU9yclhNSk8lWW1QZXZETZY0Z2ZXWnFZTkdRYmVXci9qTFlyVD1NR2NERUVpQkNBUz
QyQjBRNVo3WXBJQ1NTZlVKZG9SZitvNXdqSVI5Tnpu0XRBSnpZZjUyUnRTVEw2M3FBU0Q1NS8wTUdPaGZweTVZZzFmcDFJSU9qSE84b3JmenBCL2S0UVVKTEdyNjl6alYxMZwyR
lZtVloZV2IZT3NsZmtHeC8rSmducFZRL2NvcVVVRW1P0FZZUHFSZzdraVVtNGZreXNAMkZNalYvUmJCQ3RFU21PV095QytJM1BuSUtjSWZrMkNvajVxMipjdzO9In0.LTbKnZxB
HLCO3T4KEzmQo8SLRvLFHlqGPqH0hFy_Cweb29R3Qt9WsZlXsyOMDA5uuxz7RZQG9Aw-jS5oGovLTTgGBkpm0_s93o8s0ydZkwekoRNmpRatiGqXwdos0C4H5nVUO_qVcijzoFf
3hUrLWBbMZGPGnuRv0qDnZX_LFPiQ9t01Ebsw6HTq0DvK5Y46wh78TRlRtrmui8UQHW9d6ZBlotEDcxlhBXgBr4XSTYo3hpgAJkNJWo4o6yE5gEAxBwgI3suUfeBG875X8FUCoC
inX7Z4IUimz5qLYd83eSElCdNfoiOXXQqeRtcKdEzeHEVwKhAefZmTK3-55nfE3Q
Content-Length: 2747048
Sec-Ch-Ua-Platform: "Windows"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJxcm5ZbmV3RFBpdkp4ZVhyYzB3emtEQmtCbCNHNmRSR2xLaHRxQXhESzcwIiwiaWF0IjoxNzYwNDI4NDIOLCJleH
Ai0jE3NjA0MzIwNDMxImFlZCI6IjAwMDAwMDAzLTAwMDAtMDAwMCljMDAwLTAwMDAwMDAwMDAwMCIsInRpZCI6ImQ3NzU4ZThmLTFkZjMtNDg5Zi04NmI1LWEyMjU0ZjUlZjljY
yIsImFwcGlkIjoiNGMxYmE5NCYtZTYxZS000TA3LTkyZDQtYTF1YjA4MGUwNGFiIiwidW5pcXV1X25hbWUiOiJwcmFzYWQubWFsYWRrYXJAYXNjZW5kaW9uLmNvbSIsInVzZXJE
ZXRhaWxzIjoidE1WalVxSVZZRkINalVSaUw1TGpUREtCL1NtRzhkN0x2Skx0dHFJYU4yZnJabzglNE1tdTJOaUFQQmFFSnV5TWNETm54NkxNelhPWmJNaFdLaUUrWHVFdERpaZZ
lZDhZc0F4YlBQTFJzc01YSWNWSWd5VUNqTzc5WFplSEZtaUNtcG8zdldWTm5ZUFhDNjc3WDYvSnhwakxIV2liaOxIOXZTUVFwZTcxeGxlR3RwVGhHUzZTZGxjSG5P0Hc2NzNZbU
lzZGFadVFzZjI5SkJDbDdZWKdONW5QeTJoZWpCLZF0MWVXSW5SUVM0ZmdNL2FTYjNFY0VHQUtWYUJXeVpuSjFrMnFNbnZ5UDJpQnlyeWNIandyTI1HUUVsSZhlK3FCenp3aO1Ja
llGWzZaY1ZTHVBkbjVlNXVFb3JxNGVqTDEvanMxMTExMWFucmFzck05SK5nKytjRjlSRW5zZ3l5RElt5VdHZ3ZzdzZvWmFNU1lOZG9nbklcRGFlZEMrR2EzNWhVcGlvTjMzVlpL
WDdYVmxFTVF0Q2dFUHJVN1VRU01RVHZ0RTJTVmIreHhhQUNjcOZV0FZmMzhRRWxnQnJZbmNlMZwzdVpZWD1Ea0IyUrdiK0lT0U5bmRVanZoNzhMRTJ4c0VkMrFUWENmS2ZZbm5
odE9PNmZTTXpDVSt0ZGRIUGr0NytOUnhDanpSVVNUT1drazS2eDVSbnlocQklJcHRMeW0rcU9yclhNSk8lWW1QZXZETZY0Z2ZXWnFZTkdRYmVXci9qTFlyVD1NR2NERUVpQkNBUz
QyQjBRNVo3WXBJQ1NTZlVKZG9SZitvNXdqSVI5Tnpu0XRBSnpZZjUyUnRTVEw2M3FBU0Q1NS8wTUdPaGZweTVZZzFmcDFJSU9qSE84b3JmenBCL2S0UVVKTEdyNjl6alYxMZwyR
lZtVloZV2IZT3NsZmtHeC8rSmducFZRL2NvcVVVRW1P0FZZUHFSZzdraVVtNGZreXNAMkZNalYvUmJCQ3RFU21PV095QytJM1BuSUtjSWZrMkNvajVxMipjdzO9In0.LTbKnZxB
HLCO3T4KEzmQo8SLRvLFHlqGPqH0hFy_Cweb29R3Qt9WsZlXsyOMDA5uuxz7RZQG9Aw-jS5oGovLTTgGBkpm0_s93o8s0ydZkwekoRNmpRatiGqXwdos0C4H5nVUO_qVcijzoFf
3hUrLWBbMZGPGnuRv0qDnZX_LFPiQ9t01Ebsw6HTq0DvK5Y46wh78TRlRtrmui8UQHW9d6ZBlotEDcxlhBXgBr4XSTYo3hpgAJkNJWo4o6yE5gEAxBwgI3suUfeBG875X8FUCoC
```


Figure 31: Multple token values with same records.

WASA 21	**Insecure HTTP Methods Allowed (PUT and DELETE)**	**Vulnerability Impact**	**Low**
		Likelihood of exploitation	**Low**
		Overall Risk Rating	**Low**
Observation	During testing, it was observed that the HTTP methods PUT and DELETE are enabled on the server or endpoint.		
Security Impact	These methods are typically used to create/replace resources (PUT) or delete them (DELETE), and if left unprotected, attackers can exploit them to: • Upload malicious files (via PUT), • Overwrite existing resources or data, • Delete critical content or configurations.		
Recommendation	Disable unused or dangerous HTTP methods (e.g., PUT, DELETE, TRACE).		
References	https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_HTTP_Methods		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	aava-uat.avateam.io		
Issue Status	**Open**		

Evidence:

Figure 13: PUT and DELETE methods are enabled.

WASA 22	Insecure Cookie Configuration	Vulnerability Impact	Medium
		Likelihood of exploitation	Low
		Overall Risk Rating	Low
Observation	During analysis, it was observed application cookie values are not configured with HttpOnly.		
Security Impact	When not configured securely, cookies can be intercepted, stolen, or manipulated, leading to security risks like session hijacking, cross-site scripting (XSS), and cross-site request forgery (CSRF).		
Recommendation	Set cookies with all secure flags with secure, HttpOnly, Samesite flags. Example: Set-Cookie: sessionid=abc123; **Secure**; **HttpOnly**; **SameSite**=Strict; Max-Age=3600		
References	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html#cookie-attributes https://owasp.org/Top10/A05_2021-Security_Misconfiguration/		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence

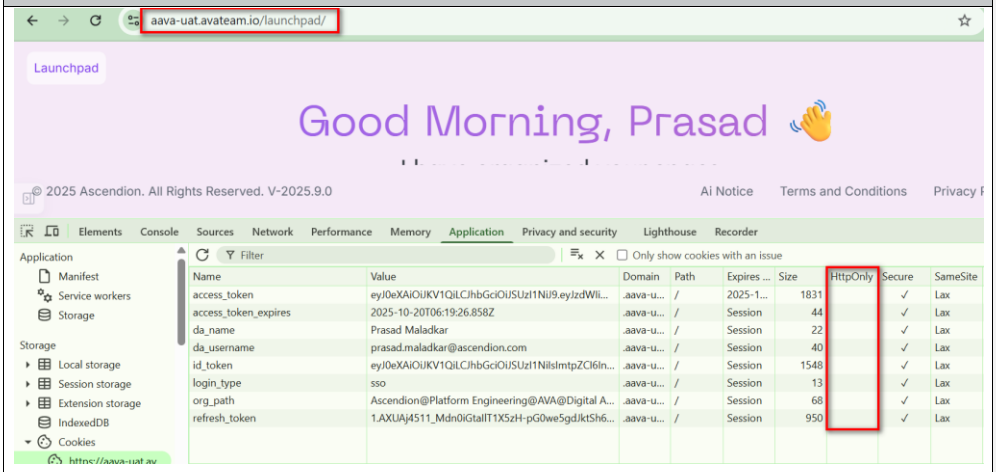


Figure 14: Cookie values do not contain HttpOnly.

WASA 23	**Cross-Origin Resource Sharing (CORS)**	**Vulnerability Impact**	**Low**
		Likelihood of exploitation	**Low**
		Overall Risk Rating	**Low**
Observation	During analysis, it was observed application that Access-Control-Allow-Origin header is configured with wildcard value (*).		
Security Impact	This misconfiguration can expose sensitive data and APIs to unauthorized third-party websites, enabling attackers to perform cross-origin requests on behalf of authenticated users.		
Recommendation	Set Access-Control-Allow-Origin to a specific list of trusted domains. • Avoid using wildcard * for credentialed requests		
References	https://cheatsheetseries.owasp.org/cheatsheets/CORS_Validation_Cheat_Sheet.html		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence

Figure 17: origin is set with wild card (*)

WASA 24	Missing preload Directive in HTTP Strict Transport Security (HSTS) Header	Vulnerability Impact	Low
		Likelihood of exploitation	Low
		Overall Risk Rating	Low
Observation	During analysis, it was observed application HSTS header is not configured with preload directives.		
Security Impact	Users may be vulnerable to man-in-the-middle (MITM) attacks during their initial connection. Without preload, users might still be exposed to SSL stripping or downgrade attacks, especially on their first visit (before the browser has seen the HSTS header).		
Recommendation	Update the HSTS header to include the preload directive, along with the required configuration. Example: *Strict-Transport-Security: max-age=63072000; includeSubDomains; preload*		
References	ttps://owasp.org/Top10/A02_2021-Cryptographic_Failures/ https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence

Figure 19: HSTS header is not configured with preload directives.

WASA 25	**X-XSS-Protection Header Disabled**	**Vulnerability Impact**	**Low**
		Likelihood of exploitation	**Low**
		Overall Risk Rating	**Low**
Observation	During analysis, it was observed application XSS protecting is disabled.		
Security Impact	The application has the X-XSS-Protection header set to 0, which explicitly disables the browser's built-in Cross-Site Scripting (XSS) protection mechanism. This setting can increase the risk of XSS attacks, especially in older browsers (like Internet Explorer and legacy versions of Chrome or Safari) that rely on this header to block reflected XSS attacks.		
Recommendation	Set X-Xss protection header security as shown below: X-XSS-Protection: 1; mode=block		
References	https://owasp.org/www-project-secure-headers/#x-xss-protection		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io		
Issue Status	**Open**		

Evidence

Figure 20: X-Xss-protection is set to 0.

WASA 26	Sensitive Information Stored in Cookies	Vulnerability Impact	Low
		Likelihood of exploitation	Low
		Overall Risk Rating	Low
Observation	During analysis, it was observed that sensitive information, including the username and email ID, is being stored in browser cookies in plaintext		
Security Impact	Storing sensitive user information in cookies without proper protection can lead to: Information Disclosure: An attacker with access to the user's browser (e.g., via XSS, shared systems, or malware) can easily read these values.		
Recommendation	Avoid storing sensitive data in cookies unless absolutely necessary		
References	https://owasp.org/www-project-secure-coding-practices/		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/console/login		
Issue Status	**Open**		

Evidence

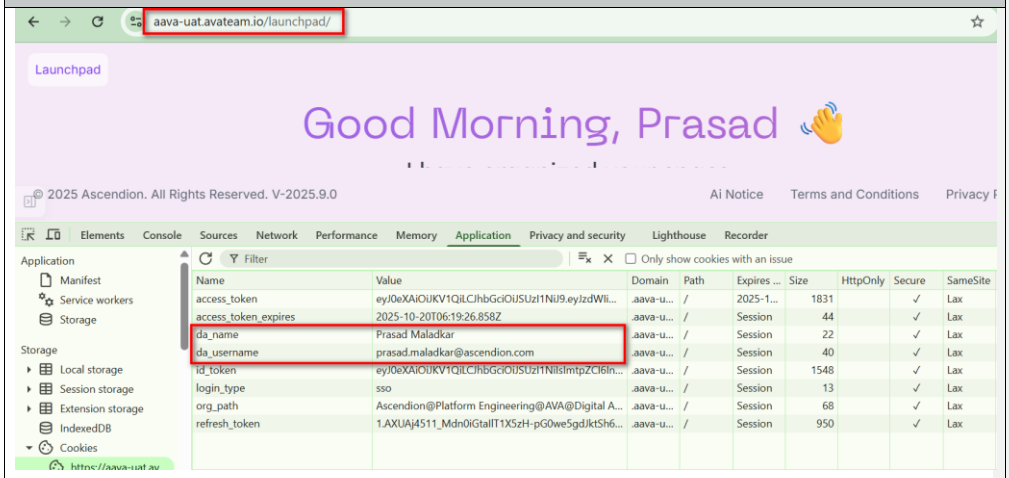


Figure 21: username and Mail Id are stored in cleartext.

WASA 27	**Missing Content Security Policy (CSP) Header**	**Vulnerability Impact**	**Low**
		Likelihood of exploitation	**Low**
		Overall Risk Rating	**Low**
Observation	The application responses do not include a Content-Security-Policy (CSP) header, leaving it vulnerable to client-side attacks such as Cross-Site Scripting (XSS).		
Security Impact	Attackers could inject or execute malicious scripts in the user's browser, leading to data theft, session hijacking, or UI redress attacks.		
Recommendation	Implement a strong CSP header restricting allowed sources for scripts, styles, and media (e.g., Content-Security-Policy: default-src 'self'; script-src 'self';).		
References	https://owasp.org/Top10/A05_2021-Security_Misconfiguration/		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/		
Issue Status	**Open**		

Evidence

Figure 21: username and Mail Id are stored in cleartext.

WASA 28	**Improper Error Handling with Source Code Disclosure**	**Vulnerability Impact**	**Low**
		Likelihood of exploitation	**Low**
		Overall Risk Rating	**Low**
Observation	The application displays detailed error messages, including stack traces and code snippets, to end-users. This discloses sensitive implementation details that could help attackers identify and exploit vulnerabilities.		
Security Impact	Detailed errors expose internal code.		
Recommendation	Implement generic user-facing error messages and log technical details only on the server.		
References	https://owasp.org/www-project-secure-coding-practices/		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/logins		
Issue Status	**Open**		

Evidence

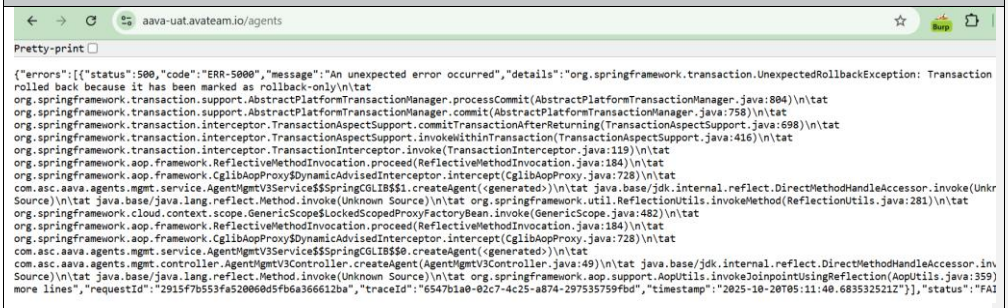


Figure 21: error message shows internal code details.

WASA 29	ETag Header Is Not Disabled	Vulnerability Impact	Informational
		Likelihood of exploitation	Informational
		Overall Risk Rating	Informational
Observation	During analysis, it was observed that the application includes **ETag (Entity Tag)** headers in HTTP responses.		
Security Impact	Information Disclosure: If ETags are derived from internal values (e.g., database IDs, timestamps, hashes of file content), attackers may infer application behavior, content changes, or file structure.		
Recommendation	Avoid generating ETags from sensitive data or user-specific content.		
References	https://owasp.org/www-project-secure-headers/#etag		
OWASP category	A05:2021 – Security Misconfiguration		
Affected URL	https://aava-uat.avateam.io/console/login		
Issue Status	**Open**		

Evidence

```
GET /698.ed3ef3c11c6635ee.js HTTP/2
Host: aava-uat.avateam.io
Origin: https://aava-uat.avateam.io
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not=A?Brand";v="24", "Chromium";v="140"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0
Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: script
Referer: https://aava-uat.avateam.io/
Accept-Encoding: gzip, deflate, br
Priority: u=1
```

```
1  HTTP/2 200 OK
2  Date: Tue, 14 Oct 2025 06:07:45 GMT
3  Content-Type: application/javascript
4  Last-Modified: Thu, 09 Oct 2025 17:26:52 GMT
5  Vary: Accept-Encoding
6  Etag: W/"68e7f05c-1adb7"
7  Expires: Wed, 14 Oct 2026 06:07:45 GMT
8  Cache-Control: max-age=31536000
9  Cache-Control: public, immutable
10 Vary: Accept-Encoding
11 Strict-Transport-Security: max-age=31536000;
   includeSubDomains
12 Access-Control-Allow-Origin: *
13 Access-Control-Allow-Credentials: true
14 Access-Control-Allow-Methods: GET, HEAD, POST, PUT,
   DELETE,PATCH, OPTIONS
15 Access-Control-Allow-Headers:
   DNT,X-Ava-Access-Token,X-Ava-Refresh-Token,Keep-Alive,User-Agent,X-Requested-With,If-Modified-Since,Cache-Control,Content-Type,Authorization,Content-Disposition,Pragma,Accept,Accept-Language
```

Figure 22: Etag is not disabled.

5.2 Conclusion

Commented [NR1]: place conclusion at the last page of the report.

In the table below, Ascendion have provided a summary of the vulnerabilities identified during the penetration testing of the application. The table presents the total number of findings categorized by their respective risk ratings—Critical, High, Medium, Low and Informational based on industry standards and internal security policies.

For a detailed explanation of the risk rating methodology, please refer to the Risk Finding section of this report.

#	Assessment	Critical	High	Medium	Low	Informational
1	AVA	9	4	7	8	1
Total		**29**				

Table 5 - Total no of observations from Pen Testing.